NIXL Cluster Topology Discovery

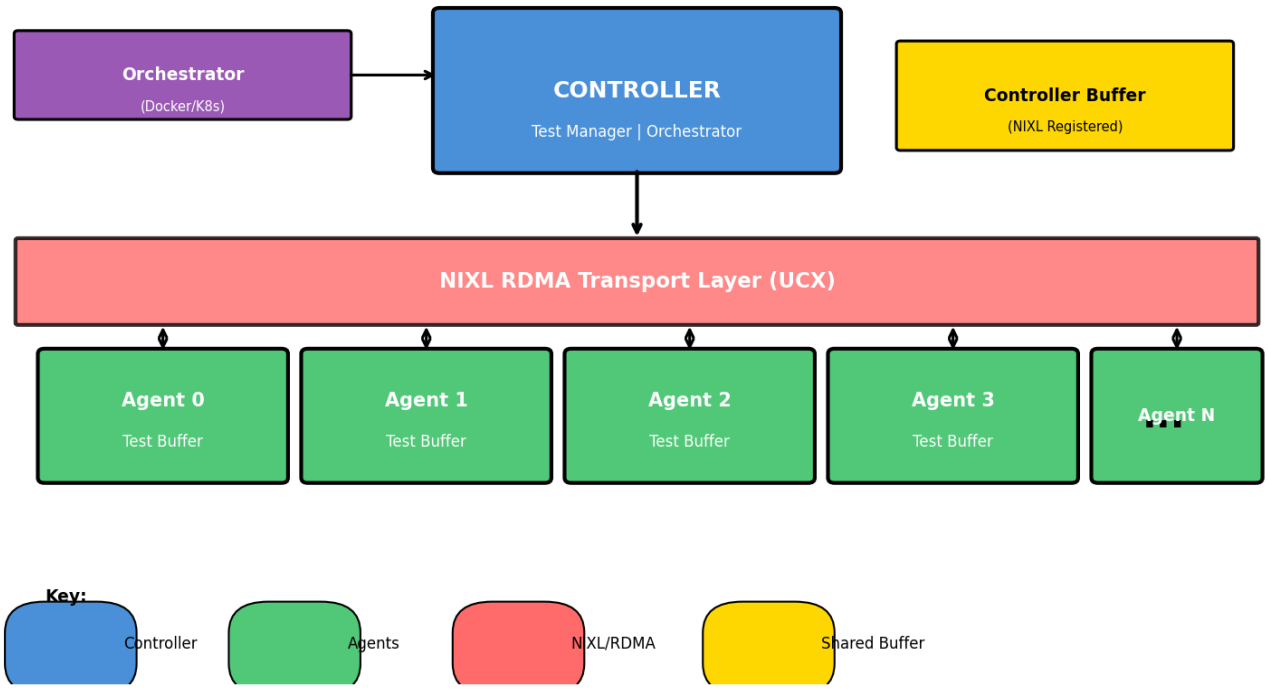
System Design Document

A distributed agent system for measuring memory transfer performance and inferring physical interconnect topology using NVIDIA NIXL framework.

1. System Architecture

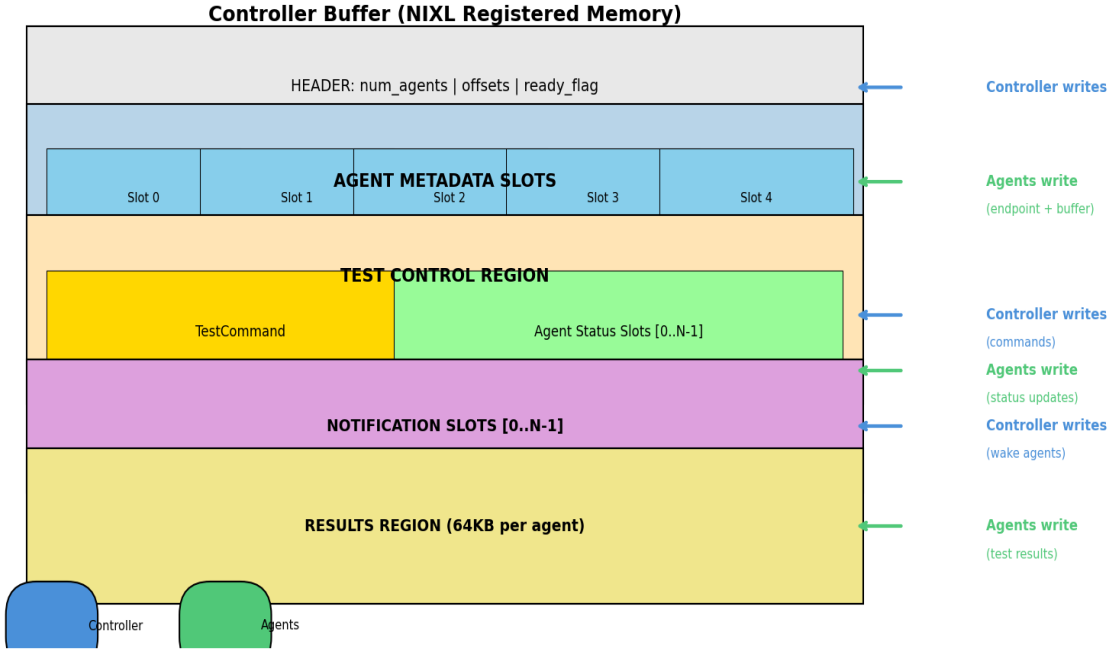
Agents run as containers spawned by the Controller via orchestrator (Docker/K8s). All communication uses NIXL RDMA transfers through a shared controller buffer - no TCP sockets. Each agent registers its NIXL endpoint during bootstrap and receives peer metadata via the controller buffer.

System Architecture - Component Overview



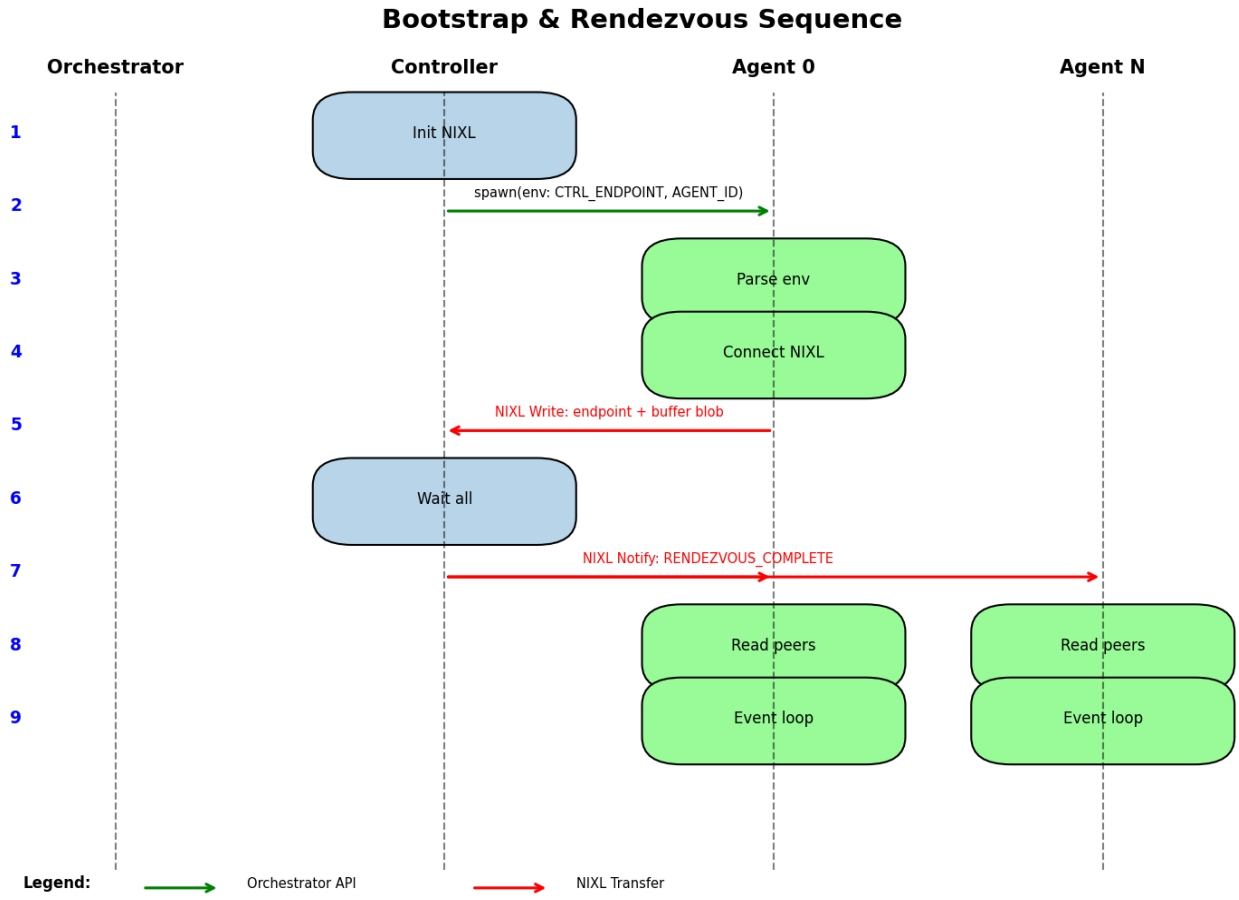
2. Controller Buffer Layout

Single NIXL-registered buffer containing all shared state: agent metadata slots for endpoint blobs, test command region for orchestration, notification slots for async wakeup, and results region for data collection. Agents compute their slot offsets by reading the header.



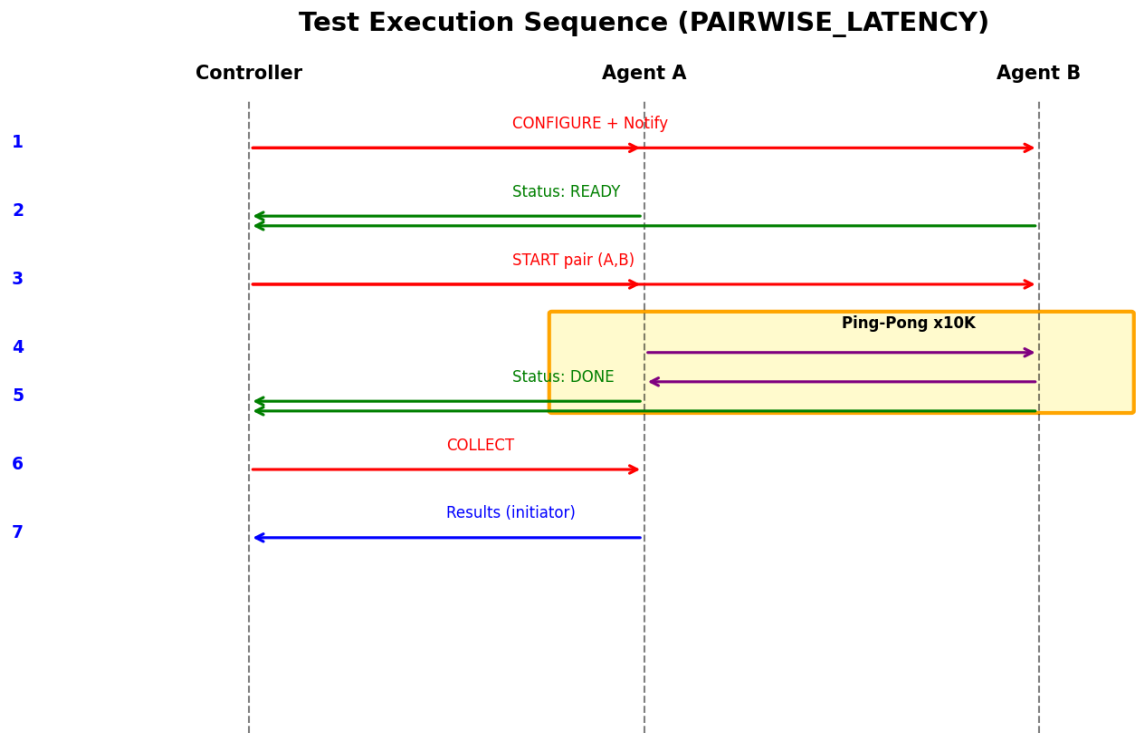
3. Bootstrap & Rendezvous Sequence

Controller spawns agent containers with NIXL metadata via environment variables (CTRL_ENDPOINT, CTRL_BUFFER, AGENT_ID). Agents decode this metadata, establish NIXL connections, and write their own endpoint info. Controller notifies all agents when rendezvous is complete.


Bootstrap & Rendezvous Sequence
Orchestrator
Controller
Agent 0
Agent N
1
Init NIXL
2
spawn(env: CTRL_ENDPOINT, AGENT_ID)
3
Parse env
4
Connect NIXL
5
NIXL Write: endpoint + buffer blob
6
Wait all
7
NIXL Notify: RENDEZVOUS_COMPLETE
8
Read peers
Read peers
9
Event loop
Event loop
Legend:
Orchestrator API
NIXL Transfer

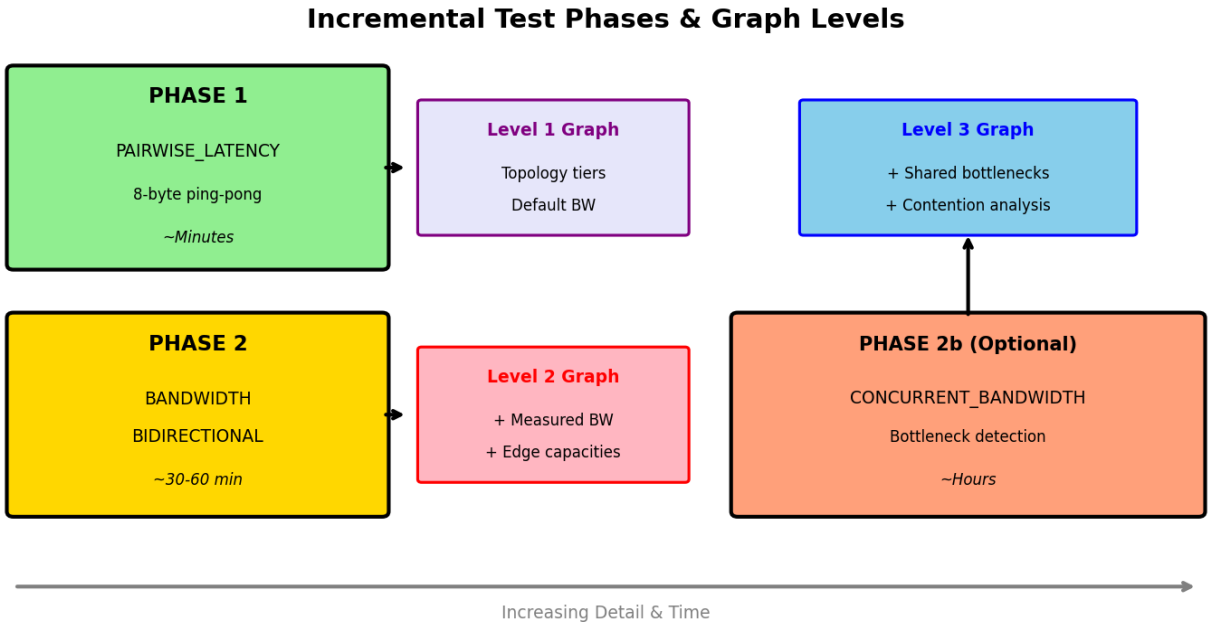
4. Test Execution Protocol

Controller writes commands to shared buffer and sends notifications. Agents wake on notification, read commands, execute tests, and update status. Notification-based design minimizes CPU polling; fallback to polling available for environments without UCX notifications.



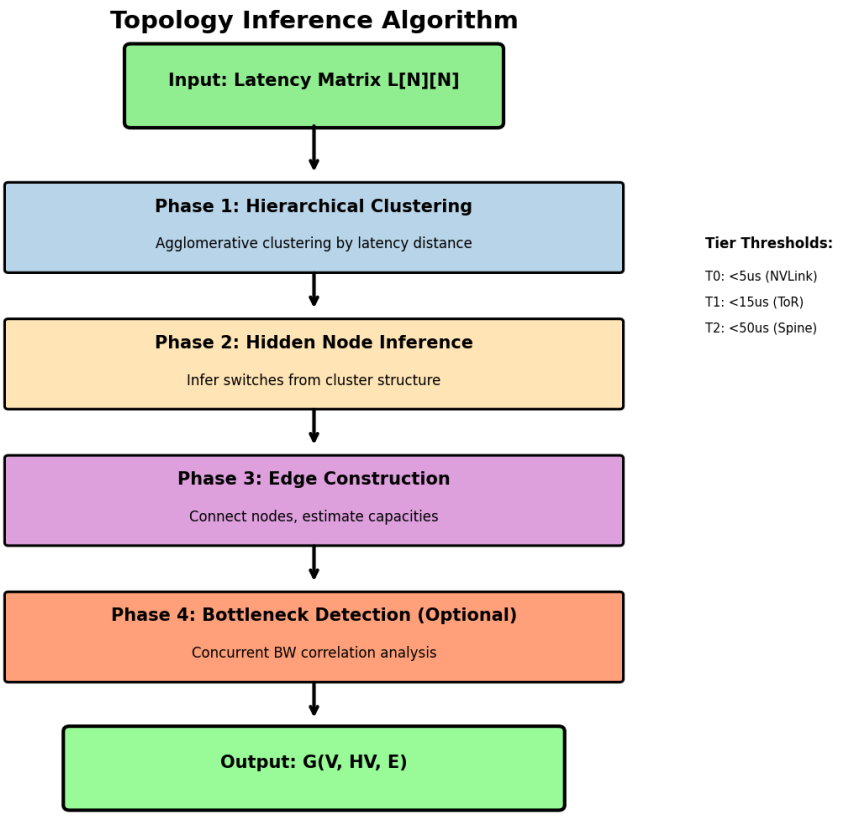
5. Incremental Test Phases

Phase 1 (PAIRWISE_LATENCY) runs quickly to produce Level 1 topology with tiers and default bandwidth. Phase 2 adds measured capacities. Optional Phase 2b detects shared bottlenecks via concurrent transfer correlation. Users can stop at any level based on time constraints.


Incremental Test Phases & Graph Levels
PHASE 1
PAIRWISE_LATENCY
8-byte ping-pong
~Minutes
Level 1 Graph
Topology tiers
Default BW
Level 3 Graph
+ Shared bottlenecks
+ Contention analysis
PHASE 2
BANDWIDTH
BIDIRECTIONAL
~30-60 min
Level 2 Graph
+ Measured BW
+ Edge capacities
PHASE 2b (Optional)
CONCURRENT_BANDWIDTH
Bottleneck detection
~Hours
Increasing Detail & Time

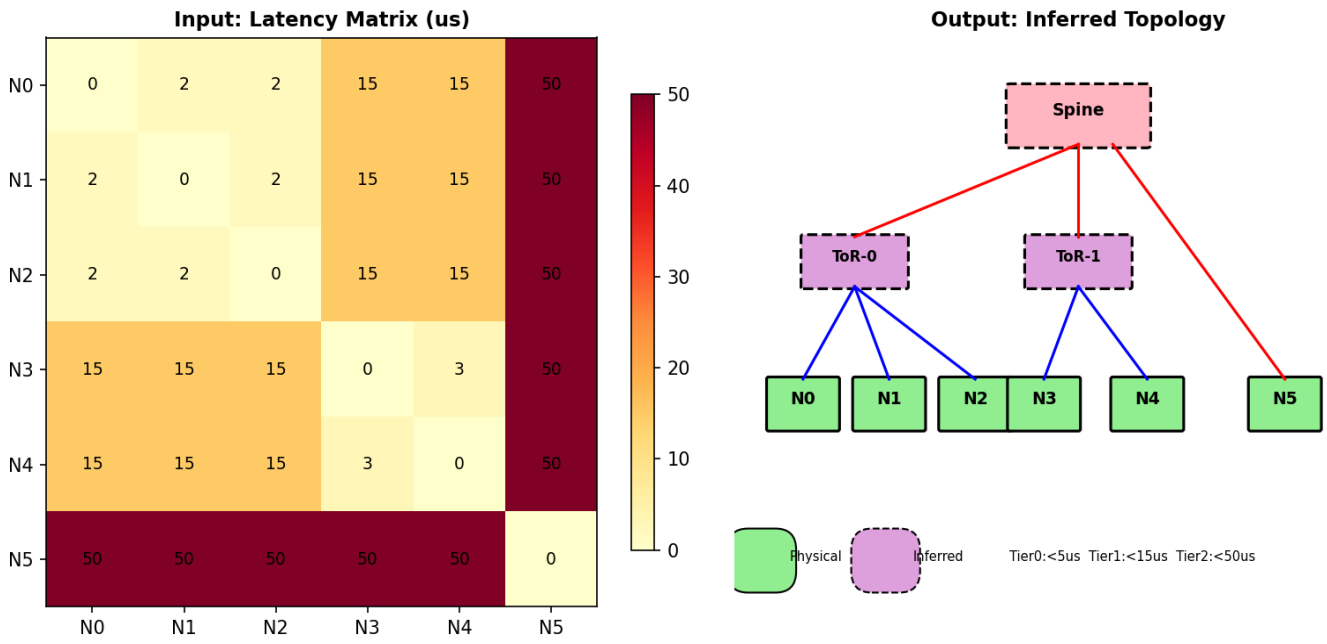
6. Topology Inference Algorithm

Four-phase algorithm: (1) Hierarchical clustering groups nodes by latency similarity, (2) Hidden node inference creates switch nodes for clusters, (3) Edge construction connects physical to hidden nodes with capacity estimates, (4) Optional bottleneck detection identifies shared links.



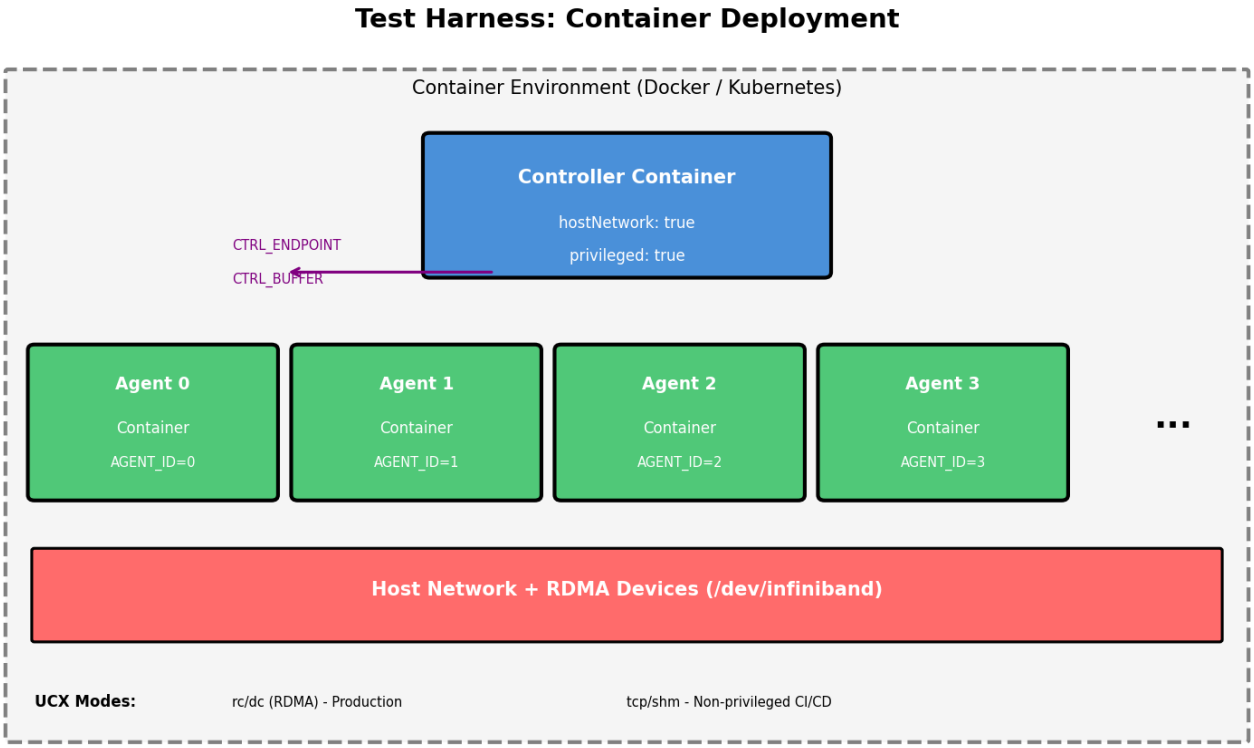
7. Topology Example: Input to Output

Example: 6-node cluster with two tight clusters (N0-N2 at 2us, N3-N4 at 3us) and one isolated node (N5 at 50us). Algorithm infers ToR switches for each cluster and spine switch connecting them, producing hierarchical graph.


Input: Latency Matrix (us)
N0 0 2 2 15 15 50
N1 2 0 2 15 15 50
N2 2 2 0 15 15 50
N3 15 15 15 0 3 50
N4 15 15 15 3 0 50
N5 50 50 50 50 50 0
N0 N1 N2 N3 N4 N5
0 10 20 30 40 50
Output: Inferred Topology
Spine
ToR-0
ToR-1
N0 N1 N2 N3 N4 N5
Physical
Inferred
Tier0:<5us Tier1:<15us Tier2:<50us

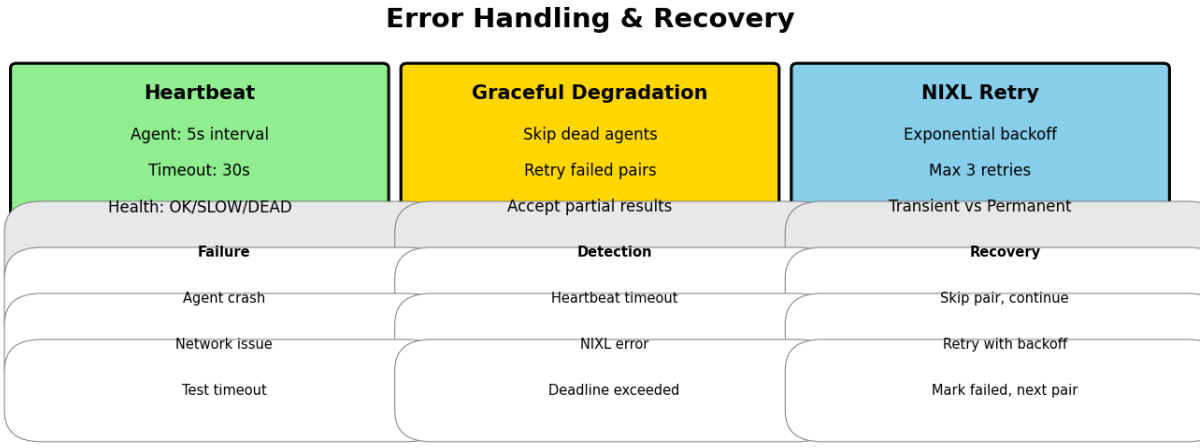
8. Test Harness Deployment

Containers require host networking and RDMA device access for production measurements. UCX transport can be configured via UCX_TLS environment variable: use 'rc/dc' for RDMA, 'tcp/shm' for non-privileged CI/CD testing. Functional tests work without special permissions.



9. Error Handling & Recovery

Agents maintain heartbeat timestamp; controller detects dead agents via timeout. Failed pairs trigger configurable retry with exponential backoff. Graceful degradation skips unavailable agents and accepts partial results rather than aborting entire test.



10. Implementation Roadmap - Next Steps

Immediate Priorities:

• Implement Agent bootstrap: env var parsing, NIXL initialization, metadata upload to controller buffer

• Implement Controller buffer management: allocation, header initialization, rendezvous wait loop

• Implement PAIRWISE_LATENCY test: ping-pong protocol, timing measurement, results aggregation

• Build basic container harness with Docker Compose for single-host testing

Phase 2 (After Core Works): Add BANDWIDTH tests, notification-based coordination, hierarchical clustering for topology inference, error handling with heartbeats and graceful degradation.

Key Design Decisions Summary

Decision	Choice	Rationale
IPC Mechanism	NIXL-only (no TCP)	Simpler security model, container-friendly
Agent spawn	Orchestrator + env vars	No handshake needed, metadata pre-shared
Coordination	Notification-based	Low CPU, immediate wakeup, scalable
Buffer allocation	Upfront (256MB/agent)	Single metadata share, no per-test sync
Topology inference	Incremental levels	Quick results first, detail as needed
Results upload	Initiator-only	Reduces data duplication